



7 janvier 2026

Achile PINSARD - Astrid MARION - Lianne SOO - Nihal LACHGUER

Tinihen MENICHE - Thomas BRUYERE

Groupe 10

Référente : Lucia BERGANTIN

Recette de projet

Conception d'un dispositif de prise de vue pour drone



IMT Atlantique

Bretagne-Pays de la Loire
École Mines-Télécom



Table des matières

1	Contexte	3
1.1	Entreprise cliente	3
1.2	Projet	3
1.3	Objectif	3
2	Méthodologie	3
2.1	Matériel utilisé	4
2.2	Outils de collaboration utilisés	4
2.3	Suivi de projet	4
2.4	Organisation du groupe	6
2.5	Documentation technique et ressources	7
3	Étude de faisabilité du projet	7
3.1	Faisabilité de la qualité photo	7
3.2	Étude de faisabilité du stockage des images	9
3.2.1	Méthode 1 : Module SD en SPI	9
3.2.2	Méthode 2 : Clé USB	10
3.2.3	Méthode 3 : Stockage interne	11
4	Analyse et solutions	11
4.1	Conclusion de faisabilité	11
4.2	La simulation de l'impulsion	12
4.2.1	Émetteur	12
4.2.2	Récepteur	12
4.3	La communication de la Raspberry vers la caméra	12
4.4	Optimisation du temps entre l'impulsion et le stockage	13
5	Résultats	15
5.1	Schémas explicatifs	15
5.2	Choix d'une caméra adaptée	17
5.3	Choix du format de photo	18
5.4	Cas d'utilisation du dispositif	18
5.4.1	Solution 1 : Commandes terminal	18
5.4.2	Solution 2 : Commande terminal + stockage USB	19
5.4.3	Solution 3 : Bibliothèque native	19
5.4.4	Solution 4 (non implémentée) : Bibliothèque native + stockage USB	19
6	Conclusion	20

Résumé

Ce document présente en détail le projet commande entreprise traitant de l'intégration d'un module caméra sur un drone du CEREMA. Il a pour objectif de décrire l'organisation du projet et les solutions techniques trouvées.

1 Contexte

1.1 Entreprise cliente

Le CEREMA est un établissement public sous la tutelle des ministères en charge de la Transition écologique, de la Décentralisation, du Logement et des Risques. Il accompagne l'État et les collectivités territoriales dans l'élaboration et la mise en œuvre des politiques publiques d'aménagement, de mobilité, de gestion des risques ou encore d'adaptation au changement climatique. Dans ce cadre, le CEREMA développe l'usage des drones pour améliorer la collecte de données sur le terrain.¹

1.2 Projet

L'enjeu pour le CEREMA est de concevoir un drone de moins de 250 grammes destiné à la cartographie environnementale pour un organisme public. Cette contrainte de masse permet de bénéficier d'une plus grande liberté réglementaire. C'est dans ce cadre que notre mission apparaît, il est question de la conception d'un dispositif de prise de vue à intégrer sur ce drone, ledit dispositif ne devant donc pas dépasser 20 à 30 grammes.

1.3 Objectif

Nous devons fournir un module global permettant de prendre une photo lorsqu'une impulsion est reçue et de la stocker avec un moyen de les identifier en fonction de l'impulsion correspondante. Également trouver un module caméra en **global shutter**² capable de faire des prises de vues offrant une résolution de minimum $5cm^2$ au sol sur une altitude pouvant varier entre 30 et 120 mètres. Chaque image devra pouvoir être stockée sans perte de qualité et dans un temps très court (voir la partie 3.2). L'ensemble de ces contraintes doit d'abord être analysé en termes de faisabilité.

2 Méthodologie

1. <https://www.cerema.fr/fr/cerema>

2. Est une technologie de capteur où tous les pixels sont exposés et lus simultanément, évitant les distorsions dues au mouvement.

2.1 Matériel utilisé

Matériel	Utilisation
Carte Raspberry pi Zéro 2W	Le cœur du dispositif. Elle reçoit les impulsions de commande et pilote le module caméra pour déclencher les prises de vue
Carte Raspberry pi 2	Utilisée pour la simulation de l'autopilote. Elle génère les impulsions nécessaires au contrôle du dispositif de prise de vue.
Module caméra v3 Raspberry Pi	Caméra dédiée à la capture des images pour les tests (ce n'est pas le modèle final retenu, seulement le matériel à disposition le plus adapté)
2 cartes SD	Supports de stockage contenant les OS des 2 Raspberry et les photos capturées
Raspberry Pi Imager	Installation des OS des Raspberry

2.2 Outils de collaboration utilisés

Outil	Utilisation	Lien
VSCode	Implémentation du code sur les Raspberry via SSH	
Github	Partage des fichiers de code entre les membres de l'équipe et les Raspberry	Cliquez ici
GanttProject	Gestion de projet : suivi de l'avancée des tâches	Cliquez ici
Trello	Outil complémentaire du Gantt permettant le suivi en temps réel de l'avancée de chaque pôle avec les tâches restantes, en cours et à réaliser	Cliquez ici
Drive	Stockage des fichiers utiles en 3 parties : ➤ Organisation (réunions et livrables) ➤ Réalisations (documents produits) ➤ Ressources (ressources permettant d'avancer)	Cliquez ici

2.3 Suivi de projet

Ci-dessous la version initiale du Gantt suivie de sa version finale après modifications à la fin du projet :

2. Méthodologie

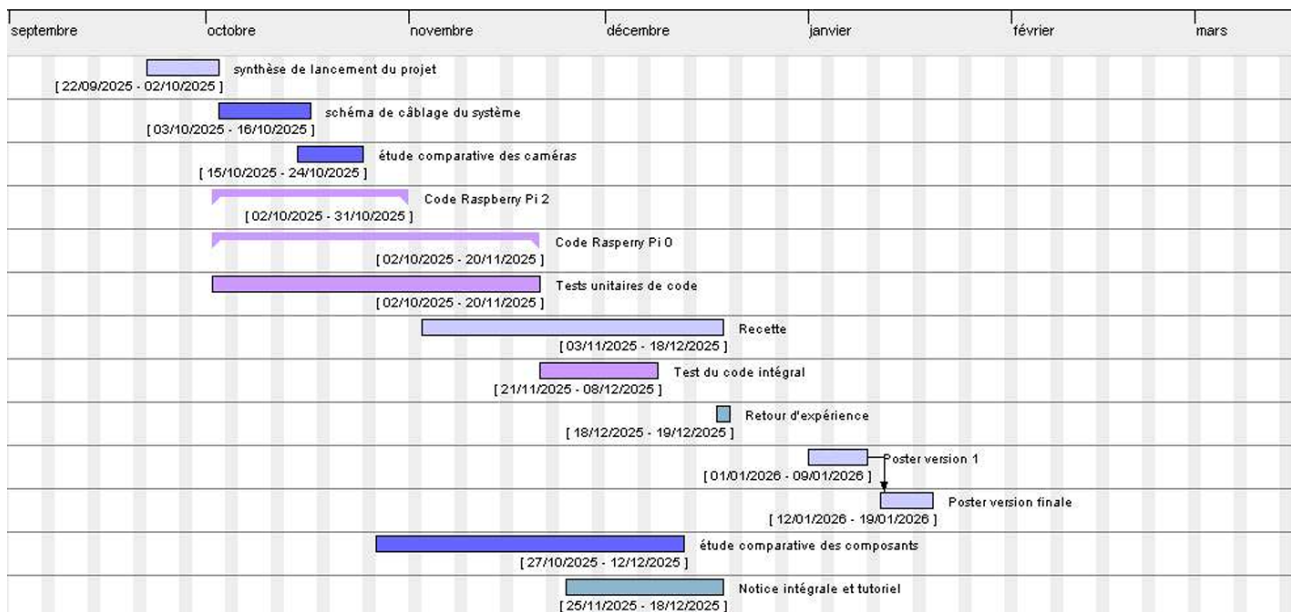


FIGURE 1 – Diagramme de Gantt version initiale

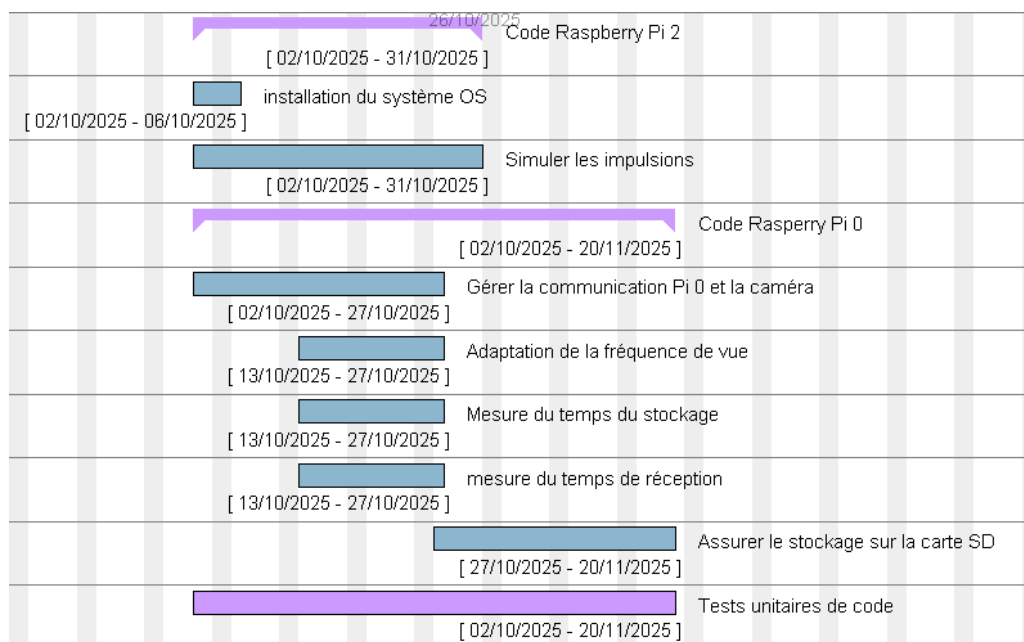


FIGURE 2 – Diagramme de Gantt version Initiale détaillée

2. Méthodologie

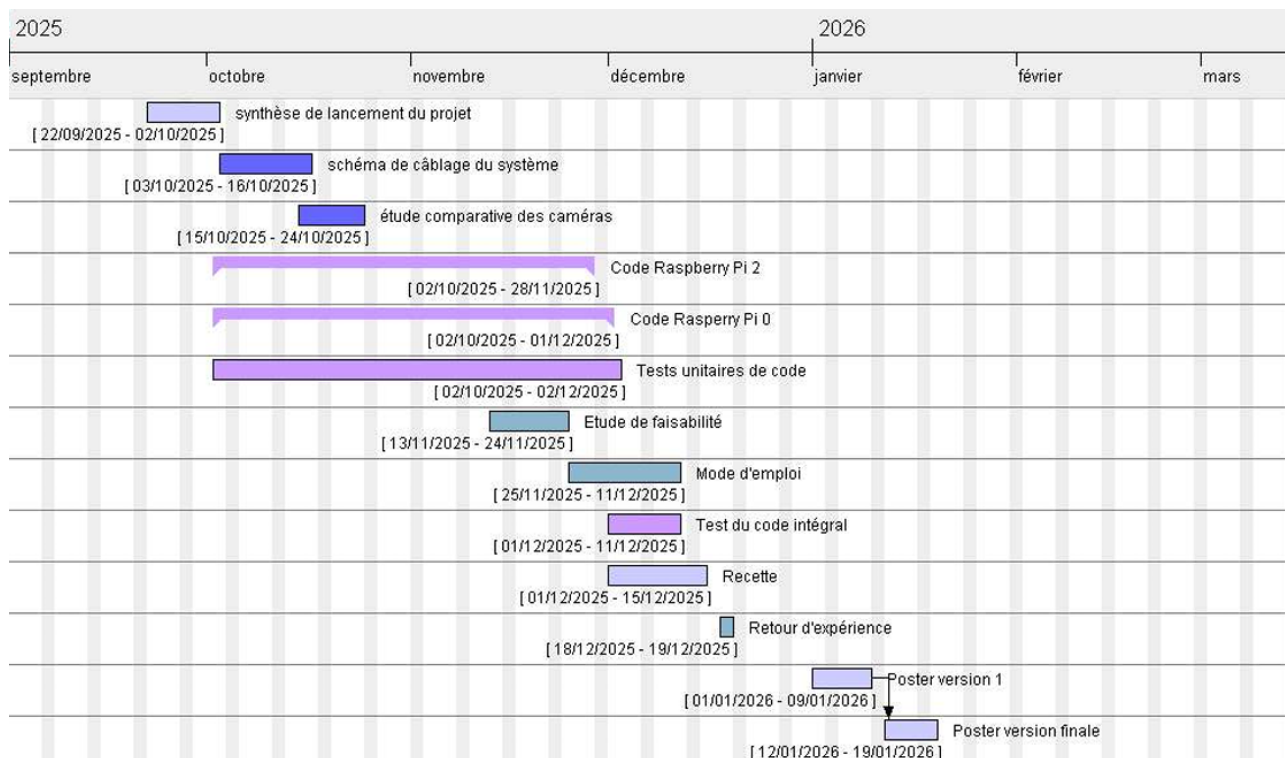


FIGURE 3 – Diagramme de Gantt version finale

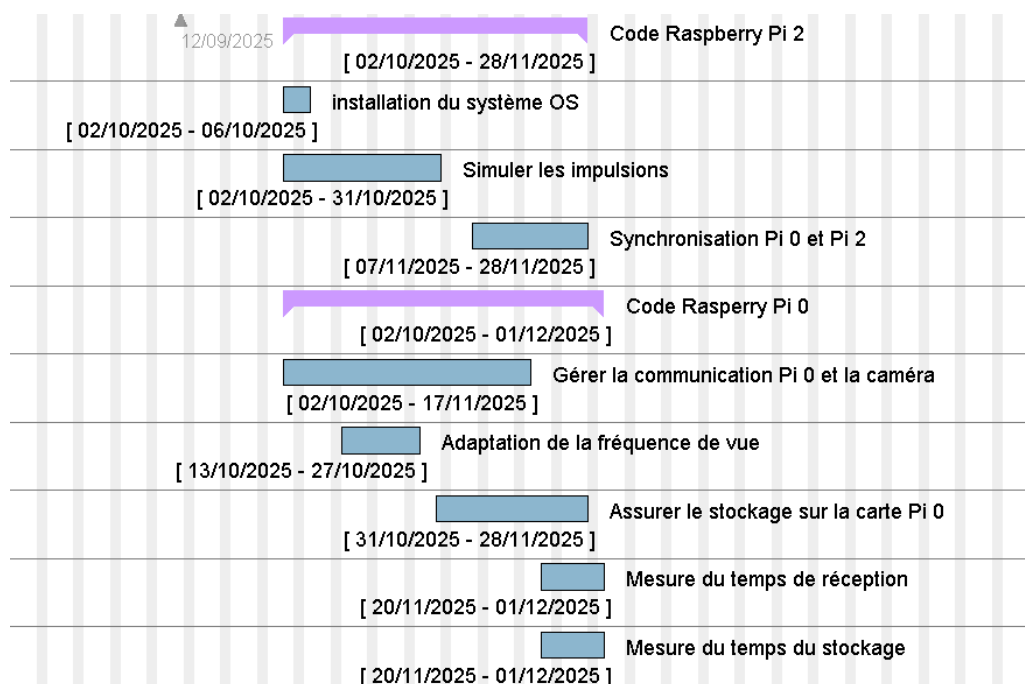


FIGURE 4 – Diagramme de Gantt version finale détaillée

2.4 Organisation du groupe

Le groupe a été réparti en sous groupes de deux personnes afin de faciliter la répartition des tâches et d'optimiser l'avancement du projet. Pour organiser efficacement le travail, nous avons identifié et isolé les

principaux enjeux techniques, qui ont ensuite servi de base à la constitution des sous-groupes. Les axes majeurs du projet sont les suivants :

- La simulation de l'impulsion
- La communication de la Raspberry vers la caméra
- La recherche de cameras
- La communication de la Raspberry vers le moyen de stockage

Comme nous travaillions tous sur le même matériel, nous voulions initialement nous organiser avec des créneaux sur la Raspberry Pi0 : un groupe possède la Raspberry pour tester ce sur quoi il travaille pendant que les autres codent, documentent, et règlent les problèmes de leur côté. Ce type de répartition pouvait amplement suffire mais à cause d'un problème d'installation de l'OS, il a été impossible de conserver cette répartition. Nous avons donc fini par prendre deux cartes Raspberry Pi0 pour avancer en parallèle.

2.5 Documentation technique et ressources

Au cours de notre projet, nous avons pu nous appuyer sur différentes ressources :

- Auto-formation via classes en ligne type OpenClassrooms pour apprendre les bases du C++
- Sites en ligne répertoriant les cas d'utilisation d'une carte Raspberry, blogs, documentation officielle de la carte et des bibliothèques.
- Vidéos Youtube, cela peut paraître enfantin mais c'est une fantastique source d'informations, on peut y retrouver des cas d'utilisation expliqués point par point et d'autres vidéos plus théoriques traitant des différents protocoles utilisés pour la communication inter-composants.
- Accompagnement personnalisé du client sur le projet : explications et propositions de solutions techniques.

3 Étude de faisabilité du projet

3.1 Faisabilité de la qualité photo

Il a été demandé d'avoir au minimum $5cm^2$ de résolution au sol, représentée sur le schéma 8. Mais cette résolution au sol va dépendre de la résolution de la caméra ainsi que de la hauteur de vol.

$$\text{Résolution}_{\text{capteur}} = \frac{4L^2}{\text{Résolution}_{\text{sol}}} \tan^2\left(\frac{\alpha}{2}\right)$$

Avec :

- $\text{Résolution}_{\text{capteur}}$ en px
- $\text{Résolution}_{\text{sol}}$ en m^2/px
- L la hauteur du drone en m

3. Étude de faisabilité du projet

► α l'angle de prise de vue en rad

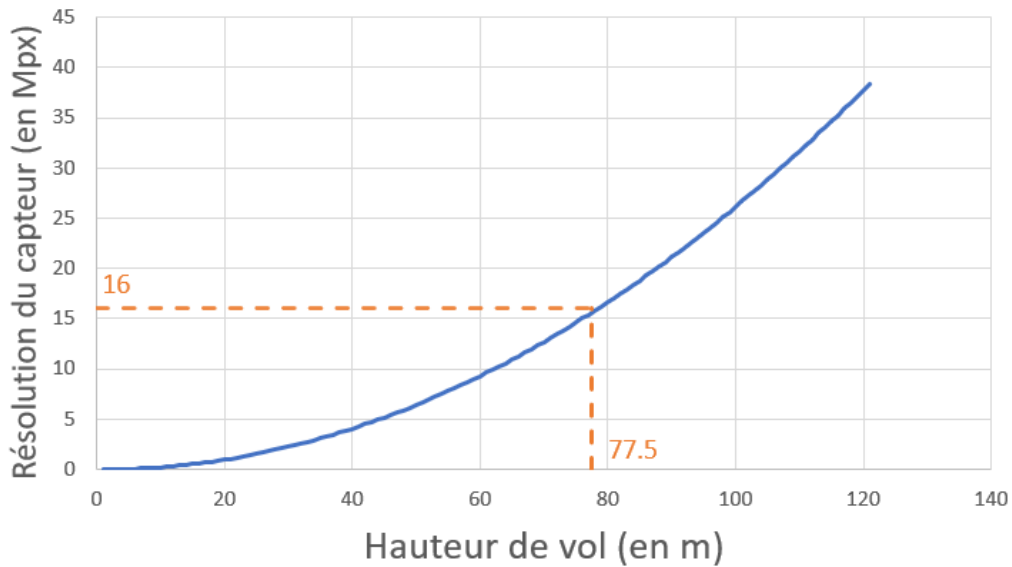


FIGURE 5 – Résolution du capteur par rapport à la hauteur de vol pour un angle α de 60° et une résolution au sol de $5 \text{ cm}^2/\text{px}$

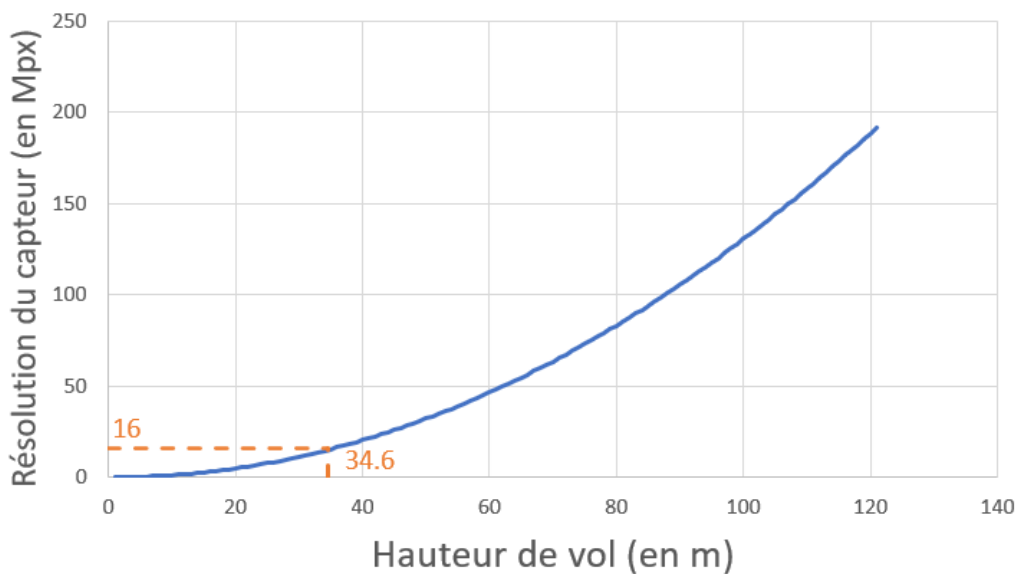


FIGURE 6 – Résolution du capteur par rapport à la hauteur de vol pour un angle α de 60° et une résolution au sol de $1 \text{ cm}^2/\text{px}$

Pour une résolution au sol de $5 \text{ cm}^2/\text{px}$, une résolution de capteur de 16 Mpx (taille du capteur de la solution trouvée) est correcte car elle permet d'aller jusqu'à 77.5 m d'altitude. Pour une résolution au sol de $1 \text{ cm}^2/\text{px}$, l'altitude est très limitée, puisqu'avec un capteur de 16 Mpx, on ne pourrait pas aller plus haut que 34.6 m d'altitude.

Or, le marché ne propose pas de caméra possédant un capteur de plus de 20 Mpx et correspondant aux

autres attentes (poids ...), et pour la suite (stockage notamment), un trop gros capteur risque de poser des problèmes.

Donc nous choisissons d'utiliser, en accord avec le client, une résolution au sol de 5 cm²/px, qui permet de conserver une résolution correcte tout en permettant d'atteindre une altitude élevée.

CONCLUSION : Il n'est pas possible d'atteindre les résolutions suffisantes avec un capteur global shutter pour les altitudes demandées. Néanmoins, les capteurs rolling shutter représentent une alternative avec laquelle nous pouvons atteindre des altitudes de 77 mètres.

3.2 Étude de faisabilité du stockage des images

Voici la formule utilisée pour déterminer la fréquence d'acquisition :

$$f = \frac{V}{D \cdot (1 - R)} \quad \text{où} \quad D = 2L \cdot \tan\left(\frac{\alpha}{2}\right)$$

Avec L la hauteur du drone, D la largeur du champ au sol, V la vitesse, α l'angle de la caméra et R le recouvrement de 0 à 1 (voir Figure 8).

Vitesse (km/h)	30 m	60 m	77 m	90 m
45	1.81	0.91	0.70	0.61
60	2.41	1.21	0.94	0.81
90	3.62	1.81	1.41	1.21

TABLE 1 – Fréquence d'acquisition (images/s) avec un recouvrement de 60 %

Vitesse (km/h)	30 m	60 m	77 m	90 m
45	4.52	2.26	1.76	1.51
60	6.03	3.02	2.35	2.01
90	9.05	4.52	3.52	3.02

TABLE 2 – Fréquence d'acquisition (images/s) avec un recouvrement de 80%

Le temps qui s'écoule entre la prise de vue de la caméra et la fin du stockage de la photo sur la carte SD est **majoritairement limité par le temps de stockage**.

Nous avons donc mesuré celui-ci afin de choisir une fréquence de prise de vue réaliste en fonction du temps d'enregistrement d'une photo.

La photo est en couleur, et composée de 16Mpx, dont chaque pixel est codé sur 10 bits (matrice de Bayer). Donc, avant compression, une image de 16 Mpx pèse environ :

$$taille = 16 \times 10 : 8 = \mathbf{15 \text{ MB}}$$

Nous avons évalué les fréquences de prise de vue pour 3 méthodes de stockage différentes :

3.2.1 Méthode 1 : Module SD en SPI

La première méthode consiste à brancher un module SD à la Raspberry et d'y stocker les données via SPI.

On obtient une vitesse d'écriture :

$$v_{writeSD} = 1.5MB/s$$

En considérant le modèle de caméra **Arducam 16 MP Autofocus (IMX519)**, nous pouvons calculer le **temps d'enregistrement d'une photo** grâce à la valeur de $v_{writeSD}$ trouvée précédemment.

Ainsi, une image serait enregistrée en :

$$t_{write-image} = \frac{taille}{v_{writeSD}}$$

A.N.

$$t_{write-image} = \mathbf{10\ s}$$

Il est donc essentiel, avec cette méthode, de compresser les photos en JPEG au vu de la valeur de $v_{writeSD}$. Grâce à libcamera, il est possible de choisir le taux de compression :

TABLE 3 – Taille d'une photo en fonction du taux de compression

Qualité indicative (en %)	Taux	Taille d'une photo (en MB)	Temps d'écriture de la photo (en s)	Fréquence d'acquisition max (en s^{-1})
95 (Très haute)	x5	9.6	6.4	0.1
90 (Haute)	x10	4.8	3.2	0.3
85 (Bonne)	x15	3.2	2.1	0.5
80 (Moyenne)	x20	2.4	1.6	0.6

Donc, au regard des Tables de la section 3.2, les fréquences obtenues ci-dessus ne sont pas compatibles avec une hauteur maximale de 77 m, et aucune vitesse entre 45 et 90 km/h, donc cette méthode ne semble pas viable.

3.2.2 Méthode 2 : Clé USB

La deuxième méthode consiste à stocker les images sur une clé USB branchée à la Raspberry.

Les vitesses d'écriture sur une clé USB dépendent de sa classe :

TABLE 4 – Vitesses d'écriture des clés USB par génération

Norme USB	Débit	Fréquence de prise de vue minimale
USB 2.0	3 à 10 Mo/s	0,20 images/s
USB 3.0 / 3.1 Gen 1	10 à 45 Mo/s	0,67 images/s
USB 3.1 Gen 2 / 3.2	50 à 150 Mo/s	3,33 images/s
USB 3.2 Gen 2x2	150 à 500 Mo/s	10,00 images/s
USB4	500 à 2 800 Mo/s	33,33 images/s

Donc, en théorie, et au regard de la section 3.2, une clé USB 3.1 Gen 2/3.2 serait une solution viable, permettant plusieurs combinaisons de vitesses et d'altitudes avec un bon recouvrement. De plus, les clés USB avec des vitesses supérieures permettraient de couvrir toutes les combinaisons.

3.2.3 Méthode 3 : Stockage interne

La troisième méthode consiste à stocker les images directement sur la carte SD hébergeant l'OS.

La vitesse d'écriture sur une carte SD dépend aussi de sa classe et de l'utilisation qu'on en fait (si plusieurs tâches tournent en parallèle, par exemple). Mais en théorie, on a les valeurs suivantes :

TABLE 5 – Classes de vitesse des cartes SD et usages

Classe	Vitesse d'écriture minimale	Fréquence de prise de vue théorique
Classe 2	2 Mo/s	0,13 images/s
Classe 4	4 Mo/s	0,27 images/s
Classe 6 / V6	6 Mo/s	0,40 images/s
Classe 10 / U1 / V10	10 Mo/s	0,67 images/s
U3 / V30	30 Mo/s	2,00 images/s
V60	60 Mo/s	4,00 images/s
V90	90 Mo/s	6,00 images/s

Ainsi, toujours au regard de la section 3.2, une clé USB de classe U3 / V30 ou plus permettrait à priori d'obtenir une fréquence de prise de vue suffisante.

4 Analyse et solutions

4.1 Conclusion de faisabilité

Pour conclure, la Méthode 1 ne permet pas d'obtenir des résultats suffisants pour une utilisation en photogrammétrie, car elle nécessite une compression trop importante des photos, impliquant une perte d'information

non négligeable.

Il faut donc utiliser une alternative pour diminuer le temps de stockage d'une photo.

La première alternative décrite est de **remplacer la carte SD par une clé USB**. Cela permettrait de transférer les images avec un temps de transfert plus court et donc sans recourir à une compression élevée. Cette solution ajouterait cependant d'autres contraintes :

La clé est **plus imposante et plus lourde**.

La consommation énergétique est plus importante, et pourrait impacter directement le temps de vol.

La seconde alternative proposée revient simplement à stocker les données **sur la carte SD de la carte Raspberry Pi Zéro**, puis, après le vol, à transférer les images sur une seconde carte SD pour les récupérer. Cette solution permettrait à priori d'obtenir des fréquences de prise de vue suffisantes, seulement un peu plus faibles que celles avec le stockage sur clé USB. Cette solution pourrait poser problème avec l'OS sur la carte SD en endommageant des fichiers ou faisant planter le module. Mais elle a l'avantage de ne pas ajouter de poids et d'être minimaliste.

4.2 La simulation de l'impulsion

4.2.1 Émetteur

Une Raspberry Pi 2 était programmée pour générer les signaux numériques. Elle utilisait des commandes de sortie GPIO (gpioWrite) pour alterner rapidement le niveau de tension entre HAUT (1) et BAS (0). Ceci créait deux types de signaux :

- Les Impulsions de Données (les commandes de l'autopilote).
- Le signal d'Horloge (la référence de temps).

4.2.2 Récepteur

Pooling : une boucle infinie pour vérifier à chaque itération l'état de ses broches d'entrée et incrémenter un compteur à chaque changement d'état détecté.

Interruptions On a adopté une architecture événementielle basée sur les interruptions : dès qu'il se produit une transition de l'état BAS à l'état HAUT, ou vice-versa, une interruption est générée. Cette interruption suspend instantanément ce que le CPU était en train de faire et met à jour l'horloge. Cela garantit de ne manquer aucune impulsion (car la fréquence d'acquisition d'un tel dispositif est très élevée par rapport à la fréquence du signal).

4.3 La communication de la Raspberry vers la caméra

Pour assurer la communication entre le Raspberry Pi et la caméra, nous avons d'abord recherché un système d'exploitation compatible avec les bibliothèques sélectionnées. La version que nous avons retenue est la version Lite de Raspberry Pi OS Bookworm (2024), qui s'est avérée parfaitement adaptée.

Cette version est effectivement compatible avec les bibliothèques libcamera et rpikam, permettant la prise de vue, la lecture et l'enregistrement des métadonnées des images. Au contraire, la version complète avec interface graphique de l'OS présente des incompatibilités avec l'interface MIPI CSI-2 du capteur, rendant la détection et l'utilisation de la caméra instables ou impossible. Heureusement, l'installation complète avec interface graphique n'est pas recommandée dans notre cas, car elle consomme plus de ressources mémoire et n'apporte pas d'avantages significatifs pour notre projet.

4.4 Optimisation du temps entre l'impulsion et le stockage

D'après ce que nous avons observé, le temps nécessaire entre une impulsion pour prendre un cliché et son stockage est nettement trop important, même lorsque les images sont stockées sur la carte SD de l'OS. Nous avons alors remarqué que des options supplémentaires permattaient d'optimiser certaines actions inutiles ou lentes effectuées par libcamera. En effet avant chaque prise de vue, libcamera :

- Calcule le temps d'exposition
- Calcule les couleurs
- Déplace la lentille pour l'autofocus

Ces différentes étapes peuvent être forcées pour ne pas les effectuer ou limiter leur durée. Dans un premier temps, le déplacement de la lentille peut être retiré, car nous avons remarqué que la lentille se place à l'infini à partir de quelques mètres, et nous serons à minimum 30 mètres pour les prises de vue en vol, ce qui est largement suffisant considérer l'objet à l'infini et ne pas bouger la focale. Les différents paramètres permettent une importante variation de temps en fonction de ce qui est limité. Dans nos options, nous avons également remarqué que le temps de transfert sur la carte SD est limité à 20 Mo/s, indépendamment de la qualité et de la vitesse de transfert de la carte SD. Mais il est possible d'utiliser une clé USB pour théoriquement augmenter la vitesse de transfert ; cela permet par exemple de gagner 200 ms comme indiqué dans le tableau suivant :

Configuration libcamera-still	Temps (ms)	Effet sur le pipeline
Sans options	4738	Tous les automatismes sont actifs : auto-exposition (AE), balance des blancs automatique (AWB), autofocus (AF) et affichage de prévisualisation. Chaque algorithme doit converger avant la capture.
-exposure sport	2629	Utilise un profil d'auto-exposition optimisé pour la vitesse. Réduit le nombre d'itérations nécessaires à la convergence de l'exposition.
-immediate	6014	Déclenche immédiatement la capture sans attendre la convergence complète, mais impose une initialisation complète du pipeline ISP, ce qui peut augmenter la latence globale.
-awb auto	2573	Active la balance des blancs automatique avec une convergence rapide. L'analyse colorimétrique est simplifiée par rapport au mode par défaut.
-shutter 1000 -gain 2.0	2723	Désactive totalement l'auto-exposition. Les paramètres d'exposition sont imposés, supprimant toute phase de convergence AE.
-autofocus-mode manual -lens-position 0.0	3086	Désactive l'autofocus. Aucun déplacement mécanique de la lentille ni calcul de netteté n'est effectué.
-exposure sport -awb auto	2442	Réduction combinée des calculs AE et AWB, limitant le nombre d'images d'analyse avant la capture finale.
-exposure sport -awb auto -autofocus-mode manual -lens-position 0.0	2463	Suppression de la plupart des algorithmes itératifs (AE, AWB, AF), laissant uniquement le traitement minimal du capteur et de l'ISP.
-exposure sport -awb auto -autofocus-mode manual -lens-position 0.0 avec écriture sur clé USB	2272	Réduction du temps d'écriture grâce à un support de stockage plus rapide que la carte SD.

TABLE 6 – Impact des paramètres libcamera-still sur le temps de capture et de stockage d'une image DNG

La différence de temps entre chaque commande est très importante : nous pouvons diviser par deux le temps nécessaire, mais même pour le minimum (2272 ms), cela reste trop élevé (fréquence maximale de 0.44 images/s). Une solution supplémentaire est néanmoins envisageable : la bibliothèque native. En effet, libcamera-still étant un programme externe, à chaque capture il :

- démarre le programme,
- initialise libcamera,
- configure le pipeline,
- lance les configurations de prise de vue,
- capture l'image,
- écrit sur le disque,
- ferme tout.

L'accès à libcamera par son API native permettrait d'éviter la réinitialisation des algorithmes automatiques à chaque capture, ce qui permettrait de réduire fortement le temps d'une prise de vue.

5 Résultats

5.1 Schémas explicatifs

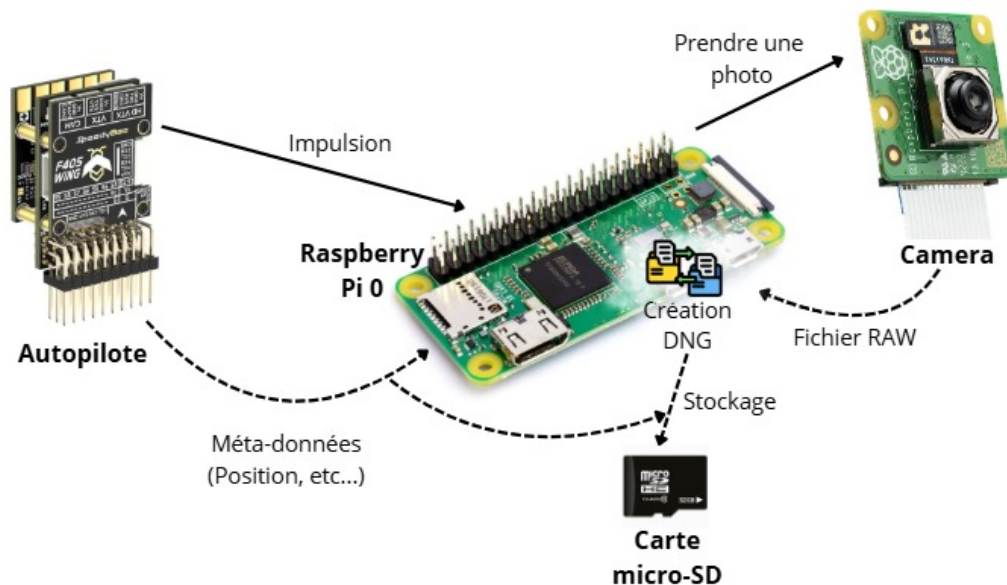


FIGURE 7 – Schéma de montage global

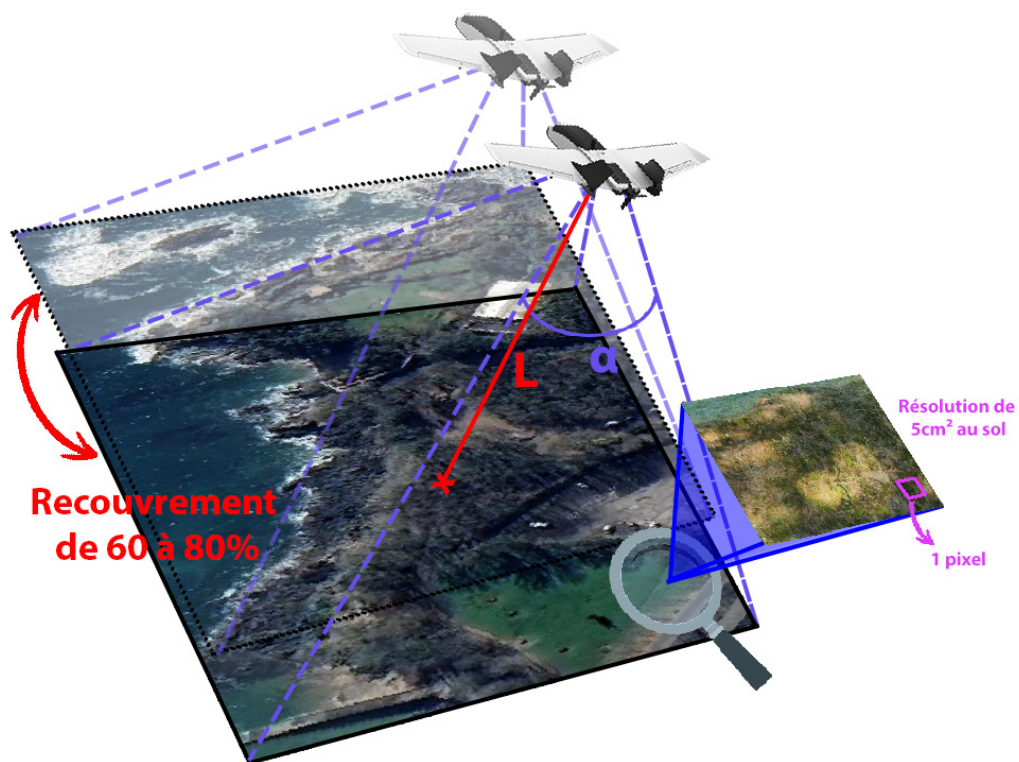


FIGURE 8 – Schéma de vulgarisation du recouvrement longitudinal et de la résolution au sol

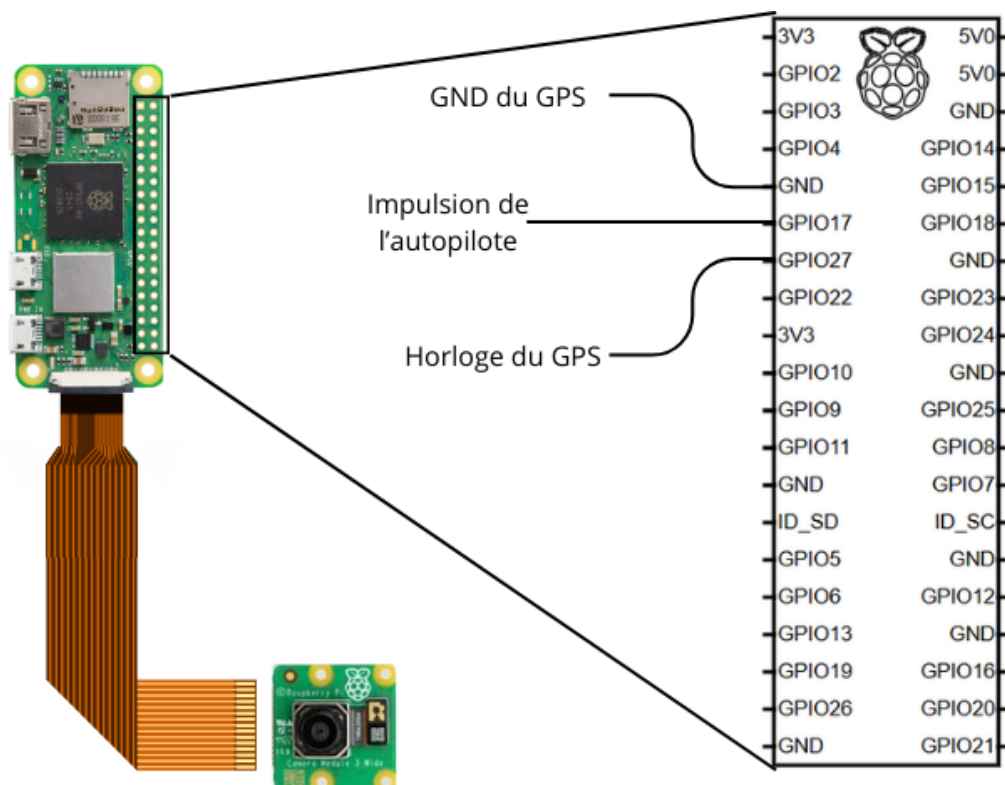


FIGURE 9 – Schéma de câblage de la carte Pi Zero

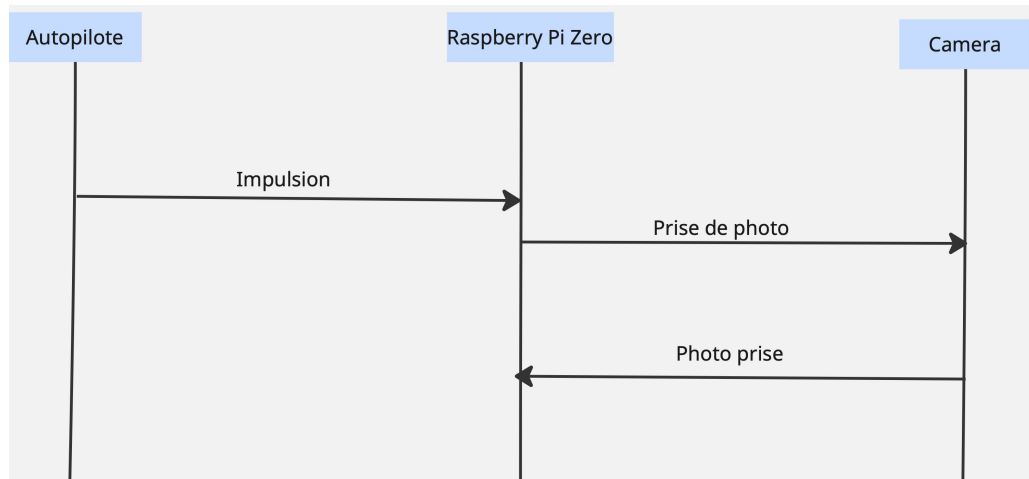
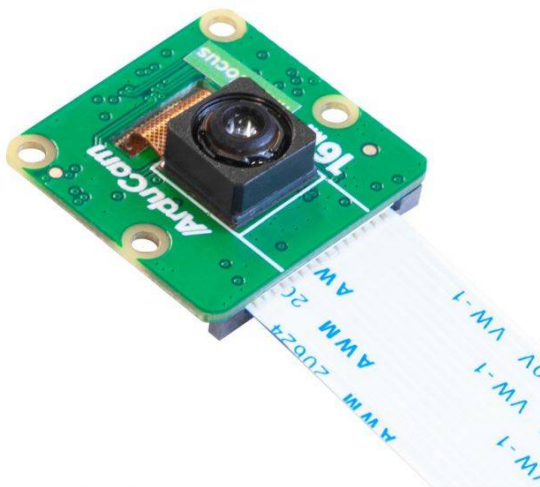


FIGURE 10 – Diagramme de séquence

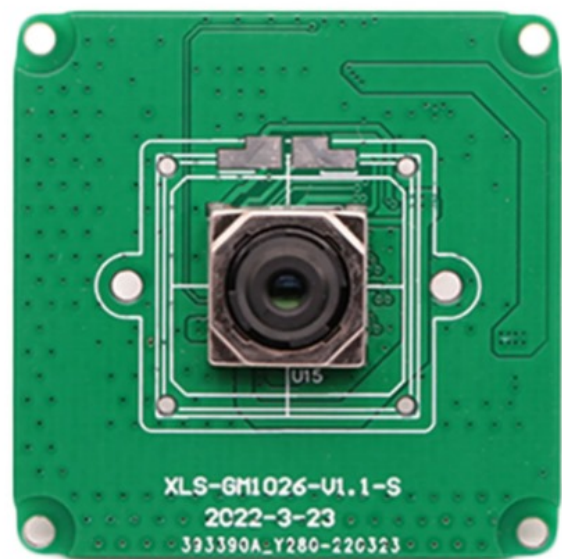
5.2 Choix d'une caméra adaptée

Après une étude comparative des caméras disponibles sur le marché, nous avons fait le choix de sélectionner les meilleurs modules. Pour plus d'informations nous vous invitons à consulter le document de cette étude comparative. Les plus intéressantes sont donc :

- l'**Arducam 16 MP Autofocus (IMX519)** car elle possède un **autofocus** et est compatible avec l'ISP de la Raspberry tout en gardant une masse acceptable et une résolution qui répond à nos attentes.
- la **SinoSense 20 MP Autofocus (IMX230)**, qui réunit une bonne résolution et un **autofocus**. Cependant, elle ne possède pas d'ISP de compression JPEG, et il reste à confirmer sa compatibilité avec la Raspberry Pi Zéro.



(a) Arducam 16 MP Autofocus (IMX519)



(b) SinoSense 20 MP Autofocus (IMX230)

FIGURE 11 – Les deux modules photos proposés

5.3 Choix du format de photo

Pour sélectionner le format optimal d'acquisition photographique, nous avons privilégié deux critères principaux : le temps de prise de vue et les métadonnées exploitables. Une comparaison systématique a été menée entre les formats JPEG, RAW et DNG.

Le format JPEG offre des fichiers compacts et des métadonnées EXIF complètes et lisibles par ExifTool. Cependant, son temps de prise de vue s'avère incompatible avec les contraintes de faisabilité du stockage identifiées dans l'étude de faisabilité, et les photos perdent en qualité à cause de la compression.

Le format RAW garantit une fidélité maximale aux données du capteur et un temps de prise de vue plus réduit, mais génère des fichiers très volumineux et présente des métadonnées limitées, rendant le post-traitement plus complexe.

Le format DNG, dérivé du RAW, offre des métadonnées complètes et lisibles par ExifTool. Ce format DNG a donc été retenu comme compromis optimal pour notre application.

5.4 Cas d'utilisation du dispositif

Le dispositif peut être utilisé de trois manières différentes. Ces trois cas correspondent à trois programmes C++. Il est possible pour chacun de les exécuter dès la mise sous tension de la carte, ce qui n'est pas le cas actuellement (nécessité d'exécuter le programme depuis un terminal). La manipulation nécessaire à cette démarche est détaillée dans la notice d'utilisation.

5.4.1 Solution 1 : Commandes terminal

La première solution utilise la carte micro-SD (celle sur laquelle se trouve le système d'exploitation) intégrée à la carte directement comme moyen de stockage. Cette décision est liée à la conclusion de l'étude de faisabilité dans laquelle nous trouvons qu'un simple module de stockage externe n'est pas suffisamment rapide.

Nous utilisons la commande `libcamera` (plus récemment appelée **rpicam**) dans un terminal basique. Cette méthode nous permet de prendre et d'enregistrer une photo de manière simple et concise, mais nécessite un temps plus long. En effet, nous n'avons pas la main sur les différentes étapes, à savoir l'initialisation, l'acquisition, le traitement et le transfert.

Afin de réduire ce temps, nous pouvons renseigner différentes options de prise de vue dont les effets sont répertoriés dans la partie "Optimisation du temps entre l'impulsion et le stockage". Les options finales utilisées sont :

- `–autofocus-mode manual –lens-position 0.0`
- `–awb auto`
- `–exposure sport`

Ces prises de vue sont effectuées par un unique programme codé en C++, et son exécution enregistre automatiquement les images dans un dossier "images" dans le répertoire courant.

L'utilisation précise de cette solution est détaillée dans la notice d'utilisation.

À l'aide de ces manipulations, nous arrivons à une fréquence d'acquisition de 0.4 images/s en moyenne (calculée sur 30 prises de photo). À noter qu'il peut y avoir une variance importante entre chaque prise de photo.

Les temps obtenus pour une image sont détaillés dans la Table 6.

5.4.2 Solution 2 : Commande terminal + stockage USB

La deuxième solution permet d'améliorer le temps de stockage en modifiant le matériel et grâce à l'utilisation d'une clé USB remplaçant le stockage interne. Cette solution permet de diminuer le temps à 2272 ms contre 2463 ms sans clé USB avec toutes les options de libcamera-still optimisées. Ainsi, cela nous permet d'obtenir une fréquence maximale de prise de vue de **0.44 images/s**, ce qui, d'après les Tables de la section 3.2 ne permet pas une performance suffisante pour de la photogrammétrie.

5.4.3 Solution 3 : Bibliothèque native

La troisième solution vise à optimiser grandement le temps de capture en utilisant la bibliothèque native. En effet, en faisant ceci, nous obtenons une maîtrise beaucoup plus bas niveau de la capture, ce qui nous permet d'accélérer certaines étapes. Ainsi, l'initialisation de la caméra est maintenant unique, il n'y a donc plus de temps d'échange entre la caméra et la carte avant chaque prise de photo pour transmettre les paramètres. Cela signifie également que ces paramètres seront les mêmes pour chaque photo, réduisant les inégalités de luminosité et de contraste.

De plus, l'étape de traitement est également reportée : nous enregistrons le flux raw simple et évitons l'étape de débayerisation qui peut être coûteuse en temps et qui peut s'effectuer en post-traitement.

Ainsi, ce sont maintenant des fichiers .raw qui sont enregistrés dans le dossier images, accompagnés de fichiers .raw.meta (comportant quelques métadonnées sur l'image). Une fois ce dossier récupéré, il est nécessaire de traiter ces images avec un programme python afin d'obtenir le format .dng analogue aux étapes précédentes (cela reste encore à implémenter).

À l'aide de ces manipulations, nous arrivons à une fréquence d'acquisition de 1.7 images/s, avec une variance relativement faible, ce qui nous donne donc une solution viable pour le projet.

5.4.4 Solution 4 (non implémentée) : Bibliothèque native + stockage USB

La dernière solution possible pour optimiser la fréquence d'acquisition serait d'utiliser la solution de stockage USB avec l'utilisation de la bibliothèque native. En effet, la fréquence trouvée dans la dernière solution semble être limitée par la vitesse d'enregistrement de la carte SD (20 Mo/s pour des images de 13Mo d'où la période de 600/700 ms).

Avec l'utilisation de la clé USB, il pourrait (nous ne ferons pas d'affirmation sans tests préalables) être possible de dépasser une fréquence d'acquisition de 2 Hz.

6 Conclusion

Pour conclure, ce projet nous a conduits à développer plusieurs solutions pour obtenir un module caméra intégrable sur le drone. Après plusieurs essais et tests, la troisième solution avec stockage en local et utilisation de la librairie native de libcamera (ou rpicam) semble être une solution viable et efficace pour de la photogrammétrie. En effet, la fréquence de prise de vue que cette solution permet est suffisante pour voler à une altitude et une vitesse raisonnable. Cependant, il reste encore à corriger le programme de post-traitement afin de convertir les fichiers .raw et .raw.meta en .dng utilisables pour Metashape.